

《操作系统》

lab4：进程管理

一. 实验要求

练习0：填写已有实验

本实验依赖实验2/3。请把你做的实验2/3的代码填入本实验中代码中有“LAB2”,“LAB3”的注释相应部分。

练习1：分配并初始化一个进程控制块（需要编码）

`alloc_proc`函数（位于`kern/process/proc.c`中）负责分配并返回一个新的`struct proc_struct`结构，用于存储新建立的内核线程的管理信息。`ucore`需要对这个结构进行最基本的初始化，你需要完成这个初始化过程。

请在实验报告中简要说明你的设计实现过程。请回答如下问题：

- 请说明`proc_struct`中 `struct context context` 和 `struct trapframe *tf` 成员变量含义和在本实验中的作用是啥？（提示通过看代码和编程调试可以判断出来）

练习2：为新创建的内核线程分配资源（需要编码）

创建一个内核线程需要分配和设置好很多资源。`kernel_thread`函数通过调用`do_fork`函数完成具体内核线程的创建工作。`do_kernel`函数会调用`alloc_proc`函数来分配并初始化一个进程控制块，但`alloc_proc`只是找到了一小块内存用以记录进程的必要信息，并没有实际分配这些资源。`ucore`一般通过`do_fork`实际创建新的内核线程。`do_fork`的作用是，创建当前内核线程的一个副本，它们的执行上下文、代码、数据都一样，但是存储位置不同。因此，我们**实际需要“fork”的东西就是stack和trapframe**。在这个过程中，需要给新内核线程分配资源，并且复制原进程的状态。你需要完成在`kern/process/proc.c`中的`do_fork`函数中的处理过程。它的大致执行步骤包括：

- 调用`alloc_proc`，首先获得一块用户信息块。
- 为进程分配一个内核栈。
- 复制原进程的内存管理信息到新进程（但内核线程不必做此事）
- 复制原进程上下文到新进程
- 将新进程添加到进程列表
- 唤醒新进程
- 返回新进程号

请在实验报告中简要说明你的设计实现过程。请回答如下问题：

- 请说明`ucore`是否做到给每个新fork的线程一个唯一的id？请说明你的分析和理由。

练习3：编写`proc_run` 函数（需要编码）

`proc_run`用于将指定的进程切换到CPU上运行。它的大致执行步骤包括：

- 检查要切换的进程是否与当前正在运行的进程相同，如果相同则不需要切换。
- 禁用中断。你可以使用 `/kern/sync/sync.h` 中定义好的宏 `local_intr_save(x)` 和 `local_intr_restore(x)` 来实现关、开中断。

- 切换当前进程为要运行的进程。
- 切换页表，以便使用新进程的地址空间。 `/libs/riscv.h` 中提供了 `lsatp(unsigned int pgdir)` 函数，可实现修改SATP寄存器值的功能。
- 实现上下文切换。 `/kern/process` 中已经预先编写好了 `switch.S`，其中定义了 `switch_to()` 函数。可实现两个进程的context切换。
- 允许中断。

请回答如下问题：

- 在本实验的执行过程中，创建且运行了几个内核线程？

完成代码编写后，编译并运行代码： `make qemu`

扩展练习 Challenge:

1. 说明语句 `local_intr_save(intr_flag); ... local_intr_restore(intr_flag);` 是如何实现开关中断的？

2. 深入理解不同分页模式的工作原理（思考题）

`get_pte()`函数（位于 `kern/mm/pmm.c`）用于在页表中查找或创建页表项，从而实现对指定线性地址对应的物理页的访问和映射操作。这在操作系统中的分页机制下，是实现虚拟内存与物理内存之间映射关系非常重要的内容。

- `get_pte()`函数中有两段形式类似的代码，结合sv32, sv39, sv48的异同，解释这两段代码为什么如此相像。
- 目前`get_pte()`函数将页表项的查找和页表项的分配合并在一个函数里，你认为这种写法好吗？有没有必要把两个功能拆开？

二. 实验流程

我们先将LAB2中`default_pmm.c`和LAB3中`trap.c`补充完整，随后开始练习实验代码补充：

练习1

补充完整的`alloc_proc`函数：

```
static struct proc_struct *
alloc_proc(void)
{
    struct proc_struct *proc = kmalloc(sizeof(struct proc_struct));
    if (proc != NULL)
    {
        //新
        // LAB4:EXERCISE1: initialize the brand-new PCB to a known state
        proc->state = PROC_UNINIT;
        proc->pid = -1;
        proc->runs = 0;
        proc->kstack = 0;
        proc->need_resched = 0;
```

```

    proc->parent = NULL;
    proc->mm = NULL;
    memset(&(proc->context), 0, sizeof(struct context));
    proc->tf = NULL;
    proc->pgdir = boot_pgdir_pa;
    proc->flags = 0;
    memset(proc->name, 0, sizeof(proc->name));
    list_init(&(proc->list_link));
    list_init(&(proc->hash_link));
}
return proc;
}

```

- `alloc_proc`: 分配一块 `struct proc_struct` 内存, 将新 PCB 填成“干净”的初始状态——状态 `PROC_UNINIT`, `pid=1`, 计数/标志全 0, 父进程指针和 `mm` 为空, `context`、`name` 清零, `tf=NULL`, `pgdir` 指向内核页表基址, 双向链表结点初始化好, 供后续挂入全局队列。

问: 请说明proc_struct中 struct context context 和 struct trapframe *tf 成员变量含义和在本实验中的作用是啥?

- `struct context context` 的作用主要有两点:

1.描述“内核线程被切出去时”的寄存器状态

每个 `proc_struct` 里有一个 `context`, 就是这个线程被调度器切走时, 留在内核栈外面、长期保存的寄存器合集。

2.被 `switch_to` 使用, 实现线程切换

在 `proc_run` 里可以看到:

```

void proc_run(struct proc_struct *proc)
{
    if (proc != current)
    {
        bool intr_flag;
        struct proc_struct *prev = current;
        local_intr_save(intr_flag);
        {
            current = proc;
            lsatp(proc->pgdir);
            switch_to(&(prev->context), &(proc->context));
        }
        local_intr_restore(intr_flag);
    }
}

```

而 `switch_to` 在 `switch.S` 里会先把当前CPU上的 `ra/sp/s0-s11` 存进 `from` 指向的 `context`; 再从 `to` 指向的 `context` 里把这些寄存器加载回来, 最后 `jr ra` 跳到对方保存的返回地址。

- `struct trapframe *tf` 的作用有:

1.当CPU发生中断/异常时, 会跳进 `_alltraps`, 在此处会把所有寄存器压栈, 整理成一个 `struct trapframe` 的布局; 同时把 `tf` 的地址作为参数, 调用 C 函数 `trap(struct trapframe *tf)`。

于是C代码里就能在 `trap()` 里, 根据 `tf->cause` 判断是中断还是异常, 转到 `interrupt_handler` 或 `exception_handler`; 以及在异常处理中访问 `tf->epc` / `tf->badvaddr` / `tf->gpr`, 打印、修改、决定返回去哪。

2.在创建新的内核线程时，内核首先在当前栈上构造一个临时的 `struct trapframe`，在其中设置将来执行入口 `kernel_thread_entry`（写入 `tf->epc`）、线程入口函数指针 `fn` 和参数 `arg`（写入通用寄存器 `s0`、`s1`），并初始化 `tf->status` 等控制寄存器字段。随后在 `copy_thread` 中将这份临时 `trapframe` 整体拷贝到子进程内核栈顶，令 `proc->tf` 指向该位置，同时将 `proc->tf->gpr.a0` 置零、`proc->tf->gpr.sp` 设置为合适的栈指针，并设置 `proc->context.ra = forkret`、`proc->context.sp = (uintptr_t)proc->tf`。当调度器首次切换到该内核线程时，通过 `switch_to` 恢复其 `context`，进入 `forkret`，再经由 `trap` 返回路径根据 `proc->tf` 中保存的寄存器值恢复 CPU 状态，从而从 `kernel_thread_entry` 开始执行该内核线程。

练习2

`do_fork`函数：

```
int do_fork(uint32_t clone_flags, uintptr_t stack, struct trapframe *tf)
{
    int ret = -E_NO_FREE_PROC;
    struct proc_struct *proc;
    if (nr_process >= MAX_PROCESS)
    {
        goto fork_out;
    }
    ret = -E_NO_MEM;

    //新
    if ((proc = alloc_proc()) == NULL)
    {
        goto fork_out;
    }

    proc->parent = current;

    if ((ret = setup_kstack(proc)) != 0)
    {
        goto bad_fork_cleanup_proc;
    }

    if ((ret = copy_mm(clone_flags, proc)) != 0)
    {
        goto bad_fork_cleanup_kstack;
    }

    copy_thread(proc, stack, tf);

    proc->pid = get_pid();
    hash_proc(proc);
    list_add(&proc_list, &(proc->list_link));
    nr_process++;

    wakeup_proc(proc);

    ret = proc->pid;
}
```

```

//
fork_out:
    return ret;

bad_fork_cleanup_kstack:
    put_kstack(proc);
bad_fork_cleanup_proc:
    kfree(proc);
    goto fork_out;
}

```

- [do fork](#): 在当前进程基础上克隆出子内核线程。具体步骤：
 1. 调 `alloc_proc` 拿到 PCB, 并设置 `parent=current`。
 2. `setup_kstack` 为子线程分配内核栈; 失败则回滚。
 3. `copy_mm` 按 `clone flags` 处理地址空间 (此实验等价于共享)。
 4. `copy_thread` 把传入的 `trapframe` 和初始上下文拷到内核栈顶部。
 5. 分配新 PID, 挂入哈希表和就绪队列, `nr_process++`。
 6. 置状态为 `PROC RUNNABLE` 并 `wakeup_proc`, 返回子进程 PID。任何一步失败按标签清理栈、PCB。

问：请说明ucore是否做到给每个新fork的线程一个唯一的id？请说明你的分析和理由。

是的。`get_pid()` 维护全局静态 `last_pid`, 每次从 `last_pid+1` 开始尝试, 在 `proc_list` 中遍历所有现存进程, 若发现某个 `proc->pid` 等于当前候选值, 就继续递增并重新检查; 只有当候选PID不与任何现存进程冲突时才返回。并且有 `static_assert(MAX_PID > MAX_PROCESS)` 保证PID空间大于最大进程数, 所以在当前所有进程都在 `proc_list` 中登记的前提下, 新分配的pid在该时刻不会与任何其他进程重复。PID可能在进程退出、从 `proc_list` 删除之后被重用, 但不会与“仍在运行/存在”的进程冲突。

练习3

`proc_run` 函数:

```

void proc_run(struct proc_struct *proc)
{
    if (proc != current)
    {
        //新
        bool intr_flag;
        struct proc_struct *prev = current;
        local_intr_save(intr_flag);
        {
            current = proc;
            lsatp(proc->pgdir);
            switch_to(&(prev->context), &(proc->context));
        }
        local_intr_restore(intr_flag);
    }
}

```

- `proc_run`: 负责切换到目标进程:
 1. 若目标就是 `current` 则直接返回。
 2. 关中断, 保存旧指针 `prev=current`, 更新 `current=proc`。
 3. 用 `lsatp(proc->pgdir)` 切换页表, 让 CPU 看见新地址空间。
 4. 调 `switch_to(&prev->context, &proc->context)` 完成上下文切换。
 5. 开中断, 回到被切换进来的进程执行点。

问: 在本实验的执行过程中, 创建且运行了几个内核线程?

两个内核线程。

在 `proc_init` 中, 首先通过 `alloc_proc()` 手工初始化了 `idleproc`, 设置 `pid = 0`、`state = PROC_RUNNABLE`、`kstack = bootstack`, 并令 `current = idleproc`, 因此 `idle` 内核线程被创建并运行。随后调用 `int pid = kernel_thread(init_main, "Hello world!!", 0)`; 通过 `kernel_thread` → `do_fork` 又创建了一个内核线程 `initproc`, 并在之后由调度器 `schedule()` 切换运行。创建并实际运行的内核线程是 `idleproc` 和 `initproc`, 共 2 个。

扩展练习 Challenge

问: 说明语句 `local_intr_save(intr_flag); ... local_intr_restore(intr_flag)`; 是如何实现开关中断的?

在 `sync.h` 里: `read_csr(sstatus)` 读 S 模式状态寄存器 `sstatus`; 其中 `SSTATUS_SIE` 位表示 S 模式全局中断使能 (1 表示允许中断, 0 表示禁止); `__intr_save()` 检查当前 `sstatus` 中 `SIE` 位是否为 1; 如果为 1, 调用 `intr_disable()` 关中断, 并返回 1; 如果本来就关着, 什么都不做, 返回 0。 `__intr_restore(flag)`: 只有当 `flag == 1` 时才调用 `intr_enable()` 重新开中断; 如果 `flag == 0`, 说明之前就是关中断状态, 就不去动它。

即:

进入临界区时, 如果之前是开中断, 就关掉并记 `intr_flag=1`;

退出时, 只在“之前是开中断”的情况下才重新打开;

如果进入时就已经关中断, 整个过程不会意外把中断打开。

问: 深入理解不同分页模式的工作原理 (思考题)

`get_pte()` 函数 (位于 `kern/mm/pmm.c`) 用于在页表中查找或创建页表项, 从而实现对指定线性地址对应的物理页的访问和映射操作。这在操作系统中的分页机制下, 是实现虚拟内存与物理内存之间映射关系非常重要的内容。

- `get_pte()` 函数中有两段形式类似的代码, 结合 `sv32`, `sv39`, `sv48` 的异同, 解释这两段代码为什么如此相像。

`get_pte()` 里两段类似的代码分别处理多级页表的两层:

```

// 第一级
pde_t *pdep1 = &pgdir[PDX1(la)];
if (!(*pdep1 & PTE_V)) {
    // alloc_page + 清零 + 填 PTE_V
}

// 第二级
pde_t *pdep0 = &((pte_t *)KADDR(PDE_ADDR(*pdep1)))[PDX0(la)];
if (!(*pdep0 & PTE_V)) {
    // alloc_page + 清零 + 填 PTE_V
}

```

多级页表每一层干的事是同一件：用对应的 VPN 段索引表项 → 若无则分配一个新的页表页 → 清零 → 设置有效位并指向下一层。

Sv32：两级（VPN[1]、VPN[0]）→ 会有两段几乎一样的逻辑；

Sv39：三级（VPN[2]、VPN[1]、VPN[0]）→ 会有三段类似逻辑；

Sv48：四级同理，只是层数更多。

索引的位数和层数不同，但每层的访问模式完全相同，所以代码结构自然很相像。

- 目前get_pte()函数将页表项的查找和页表项的分配合并在一个函数里，你认为这种写法好吗？有没有必要把两个功能拆开？

不好。应该拆开，做成“纯查找函数”和“按需分配函数”两个接口。理由如下：

1. 职责不单一：现在的 get_pte 同时负责遍历页表、判断是否存在、在缺页表时分配新页表，这三个职责耦合在一起，不利于阅读和维护。

2. 语义不干净：调用者想“只是看看有没有PTE”，结果顺手可能就把中间页表给分配了，这在调试和错误排查时比较难追踪。

3. 拆开后更清晰：

lookup_pte(pgdir, la)：只查，绝不分配。

ensure_pte(pgdir, la)：查找，不存在就分配，中间失败明确返回错误。

接口语义更清晰。

运行结果

编译：

```

wey@wey:/mnt/d/os-riscv/labcode/lab4$ make clean
rm -f -r obj bin
wey@wey:/mnt/d/os-riscv/labcode/lab4$ make
+ cc kern/init/entry.S
+ cc kern/init/init.c
+ cc kern/libs/readline.c
+ cc kern/libs/stdio.c
+ cc kern/debug/kdebug.c
+ cc kern/debug/kmonitor.c
+ cc kern/debug/panic.c
+ cc kern/driver/clock.c
+ cc kern/driver/console.c
+ cc kern/driver/dtb.c
+ cc kern/driver/intr.c

```

```
+ cc kern/driver/picirq.c
+ cc kern/trap/trap.c
+ cc kern/trap/trapentry.S
+ cc kern/mm/default_pmm.c
+ cc kern/mm/kmalloc.c
+ cc kern/mm/pmm.c
+ cc kern/mm/vmm.c
+ cc kern/process/entry.S
+ cc kern/process/proc.c
+ cc kern/process/switch.S
+ cc kern/schedule/sched.c
+ cc libs/hash.c
+ cc libs/printfmt.c
+ cc libs/string.c
+ ld bin/kernel
riscv64-unknown-elf-ld: removing unused section '.rodata.__warn.str1.8' in file
'obj/kern/debug/panic.o'
riscv64-unknown-elf-ld: removing unused section '.text.__warn' in file
'obj/kern/debug/panic.o'
riscv64-unknown-elf-ld: removing unused section '.text.is_kernel_panic' in file
'obj/kern/debug/panic.o'
riscv64-unknown-elf-ld: removing unused section '.text.kbd_intr' in file
'obj/kern/driver/console.o'
riscv64-unknown-elf-ld: removing unused section '.text.serial_intr' in file
'obj/kern/driver/console.o'
riscv64-unknown-elf-ld: removing unused section '.text.pic_enable' in file
'obj/kern/driver/picirq.o'
riscv64-unknown-elf-ld: removing unused section '.text.slob_init' in file
'obj/kern/mm/kmalloc.o'
riscv64-unknown-elf-ld: removing unused section '.text.slob_allocated' in file
'obj/kern/mm/kmalloc.o'
riscv64-unknown-elf-ld: removing unused section '.text.kallocated' in file
'obj/kern/mm/kmalloc.o'
riscv64-unknown-elf-ld: removing unused section '.text.ksize' in file
'obj/kern/mm/kmalloc.o'
riscv64-unknown-elf-ld: removing unused section '.text.tlb_invalidate' in file
'obj/kern/mm/pmm.o'
riscv64-unknown-elf-ld: removing unused section '.rodata.print_vma.str1.8' in file
'obj/kern/mm/vmm.o'
riscv64-unknown-elf-ld: removing unused section '.text.print_vma' in file
'obj/kern/mm/vmm.o'
riscv64-unknown-elf-ld: removing unused section '.rodata.print_mm.str1.8' in file
'obj/kern/mm/vmm.o'
riscv64-unknown-elf-ld: removing unused section '.text.print_mm' in file 'obj/kern/mm/vmm.o'

riscv64-unknown-elf-ld: removing unused section '.text.mm_create' in file
'obj/kern/mm/vmm.o'
riscv64-unknown-elf-ld: removing unused section '.text.vma_create' in file
'obj/kern/mm/vmm.o'
riscv64-unknown-elf-ld: removing unused section '.text.mm_destroy' in file
'obj/kern/mm/vmm.o'
riscv64-unknown-elf-ld: removing unused section '.text.set_proc_name' in file
'obj/kern/process/proc.o'
```



```
riscv64-unknown-elf-ld: removing unused section '.text.get_proc_name' in file
'obj/kern/process/proc.o'
riscv64-unknown-elf-ld: removing unused section '.text.find_proc' in file
'obj/kern/process/proc.o'
riscv64-unknown-elf-ld: removing unused section '.text.sprintf' in file
'obj/libs/printfmt.o'
riscv64-unknown-elf-ld: removing unused section '.text.snprintf' in file
'obj/libs/printfmt.o'
riscv64-unknown-elf-ld: removing unused section '.text.vsnprintf' in file
'obj/libs/printfmt.o'
riscv64-unknown-elf-ld: removing unused section '.text.strncpy' in file 'obj/libs/string.o'
riscv64-unknown-elf-ld: removing unused section '.text.strfind' in file 'obj/libs/string.o'
riscv64-unknown-elf-ld: removing unused section '.text.strtol' in file 'obj/libs/string.o'
riscv64-unknown-elf-ld: removing unused section '.text.memmove' in file 'obj/libs/string.o'
riscv64-unknown-elf-objcopy bin/kernel --strip-all -O binary bin/ucore.img
```

运行结果：

```
wey@wey:/mnt/d/os-riscv/labcode/lab4$ make qemu
```

OpenSBI v0.4 (Jul 2 2019 11:53:53)

[illegible]

```
Platform Name      : QEMU Virt Machine
Platform HART Features : RV64ACDFIMSU
Platform Max HARTs  : 8
Current Hart       : 0
Firmware Base      : 0x80000000
Firmware Size      : 112 KB
Runtime SBI Version : 0.1
```

```
PMP0: 0x0000000080000000-0x000000008001ffff (A)
PMP1: 0x0000000000000000-0xffffffffffffffff (A,R,W,X)
DTB Init
HartID: 0
DTB Address: 0x82200000
Physical Memory from DTB:
  Base: 0x0000000080000000
  Size: 0x0000000080000000 (128 MB)
  End: 0x0000000087ffffff
DTB init completed
(THU.CST) os is loading ...
```

```
Special kernel symbols:
  entry  0xc020004a (virtual)
  etext  0xc0203e6e (virtual)
  edata  0xc0209030 (virtual)
  end    0xc020d4f0 (virtual)
Kernel executable memory footprint: 54KB
memory management: default_pmm_manager
physical memory map:
  memory: 0x08000000, [0x80000000, 0x87ffffff].
vapaofset is 18446744070488326144
check_alloc_page() succeeded!
check_pgdir() succeeded!
check_boot_pgdir() succeeded!
use SLOB allocator
kmalloc_init() succeeded!
check_vma_struct() succeeded!
check_vmm() succeeded.
alloc_proc() correct!
++ setup timer interrupts
this initproc, pid = 1, name = "init"
To U: "Hello world!!".
To U: "en.., Bye, Bye. :)"
kernel panic at kern/process/proc.c:346:
  process exit!!.

Welcome to the kernel debug monitor!!
Type 'help' for a list of commands.
```